

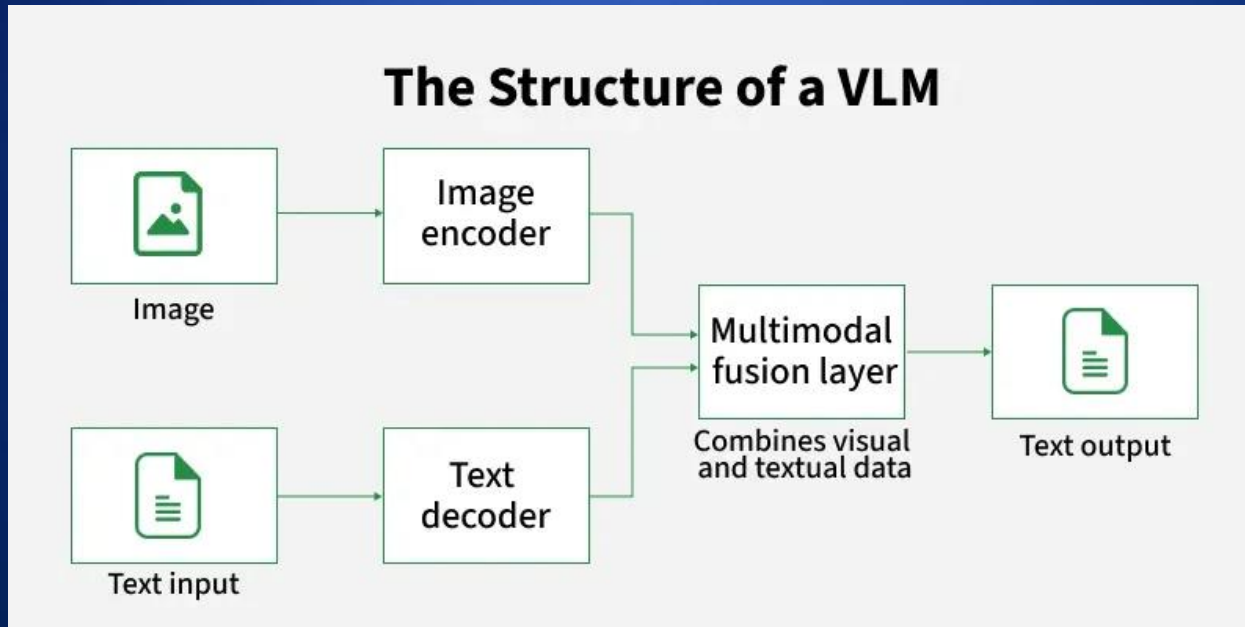


# NeuroCogniAI

**ARTIFICIAL INTELLIGENCE  
IN  
DEEP LEARNING / GENERATIVE /  
AGENTIC AI MODELS**

# VISION LANGUAGE MODEL (VLM)

Vision Language Models (VLMs) are AI systems that combine computer vision and natural language processing (NLP) to understand, interpret, and reason across both image/video and text inputs.



# TYPES OF VISION LANGUAGE MODEL (VLM)

## Types of Vision Language Models (VLMs)

### Vision-to-Text



Convert images into text

### Text-to-Vision



Create images from text descriptions.

### Cross-Modal Retrieval



Find images or text based on the other.

# APPLICATIONS OF VISION LANGUAGE MODEL (VLM)

01

Image  
Captioning

02

Visual Question  
Answering (VQA)

03

Image Search  
& Retrieval

**Applications of VLMs**

04

Content  
Creation

05

Robotics

06

Assistive  
Technologies

# COMPARISON BETWEEN LLM , VLM & VLA

| Model Type                   | Processes               | Examples             | Use Cases                               |
|------------------------------|-------------------------|----------------------|-----------------------------------------|
| LLM (Large Language Model)   | Text only               | GPT-4, BERT, LLaMA   | Chatbots, Content Generation, NLP       |
| VLM (Vision-Language Model)  | Images + Text           | CLIP, BLIP, Flamingo | Image Captioning, Visual Q&A, Search    |
| VLA (Vision-Language-Action) | Images + Text + Actions | PaLM-E, RT-2, SayCan | Robotics, AI Agents, Autonomous Systems |

# IMPLEMENTATION OF VISION LANGUAGE MODEL (VLM)

## Step 1: Import Required Libraries

- We need [PyTorch](#) for tensor operations and model inference.
- [PIL](#) is used to open and process images.
- [Transformers](#) library provides the pre-trained Qwen2-VL model and processor.
- `qwen_vl_utils` contains helper functions for processing images for the model.

```
import torch
from PIL import Image
from transformers import Qwen2VLForConditionalGeneration, Qwen2VLProcessor
from qwen_vl_utils import process_vision_info
```



# IMPLEMENTATION OF VISION LANGUAGE MODEL (VLM)

## Step 2: Load Pre-trained Model and Processor

- Specify the model ID for the Qwen2-VL model.
- Load the pre-trained Qwen2-VL model with bfloat16 for efficient GPU memory usage.
- Load the processor which will handle text and image preprocessing.

```
model_id = "Qwen/Qwen2-VL-7B-Instruct"
device = "cuda" if torch.cuda.is_available() else "cpu"

model = Qwen2VLForConditionalGeneration.from_pretrained(
    model_id,
    device_map="auto",
    torch_dtype=torch.bfloat16,
)

processor = Qwen2VLProcessor.from_pretrained(model_id)
```

# IMPLEMENTATION OF VISION LANGUAGE MODEL (VLM)

## Step 3 : Generate Function

- Loads an image and prepares it with the user query for processing by a Vision-Language Model (VLM).
- Uses the processor to format text, image, and chat structure into model-ready inputs.
- Generates a response from the model using both visual and textual information.
- Decodes the generated output into readable text and returns it as the final answer.

```
def generate_answer(image_path, query, max_new_tokens=256):
    image = Image.open(image_path).convert("RGB")
    sample = {
        "messages": [
            {"role": "system", "content": [{"type": "text", "text": "You are a Vision Language Model. Answer concisely."}]},
            {"role": "user", "content": [{"type": "image", "image": image}, {"type": "text", "text": query}]}
        ]
    }

    text_input = processor.apply_chat_template(sample['messages'][1:2], tokenize=False, add_generation_prompt=True)
    image_inputs, _ = process_vision_info(sample['messages'])

    model_inputs = processor(text=[text_input], images=image_inputs, return_tensors="pt").to(device)

    generated_ids = model.generate(*model_inputs, max_new_tokens=max_new_tokens)

    trimmed_generated_ids = [out_ids[len(in_ids):] for in_ids, out_ids in zip(model_inputs.input_ids, generated_ids)]

    output_text = processor.batch_decode(trimmed_generated_ids, skip_special_tokens=True,
                                         clean_up_tokenization_spaces=False)

    return output_text[0]
```



# IMPLEMENTATION OF VISION LANGUAGE MODEL (VLM)

## Step 4: Running the Function

- Provide the image path and the text prompt you want the VLM to answer.
- The function is executed, and the generated response is printed.

Used image is:



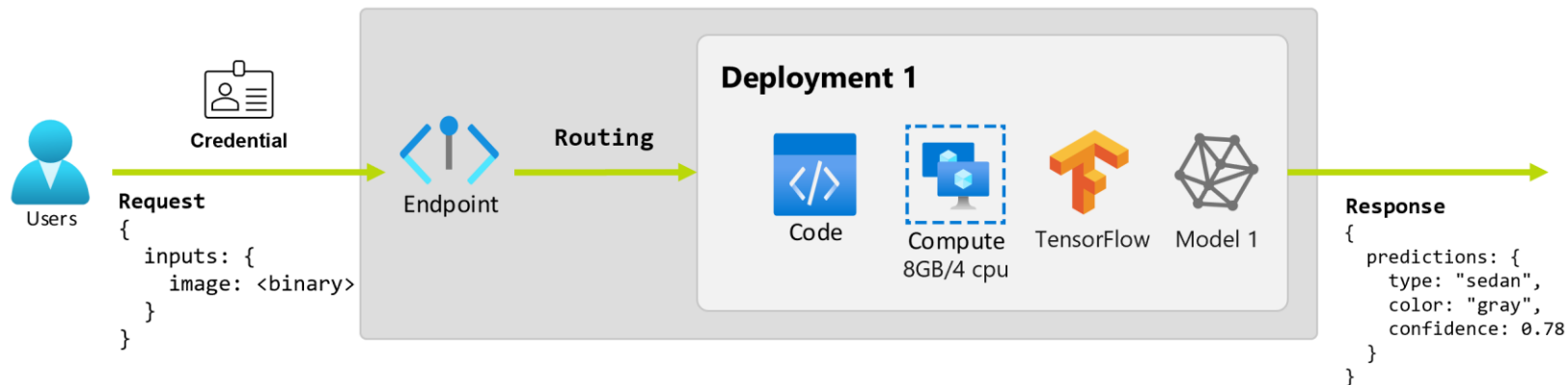
Input image

```
image_path = "/content/image.png"
query = "Describe this Image"
answer = generate_answer(image_path, query)
print("Generated Answer:", answer)
```

## Output:

*Generated Answer: The image shows a young panda bear climbing a tree. The panda has a fluffy white body with black markings on its ears, eyes, and limbs.*

# ML / DL UTILISATION USING ENDPOINTS



# ML / DL UTILISATION USING ENDPOINTS

